

Lời giải USACO theo NOCOW

Nguồn gốc: USACO Solution Notes, NOCOW edition

USACO / USACO solution notes, NOCOW edition, 47 pages

Ghi chú về nguồn. Nguồn là một tuyển tập lời giải cộng đồng, gom nhiều cách làm, nhận xét về độ mạnh của dữ liệu, và tham chiếu tới các chương trình Pascal/C/C++ đi kèm. Bản dịch này đi theo thứ tự các bài trong tài liệu gốc, giữ lại thuật toán chính, các biến thể đáng chú ý và các cảnh báo cài đặt quan trọng. Những đoạn chỉ trở tới mã nguồn hoặc bình luận về đoạn mã mẫu được diễn giải ở mức ý tưởng theo ngoại lệ code-template của dự án, thay vì chép lại toàn bộ mã.

Chương 1

Section 1.1

Your Ride Is Here (ride). Đây là bài ad hoc mở đầu. Chỉ cần đổi từng chữ cái sang giá trị, nhân các giá trị của mỗi chuỗi, lấy modulo theo yêu cầu rồi so sánh hai kết quả. Không cần thuật toán đặc biệt; điểm cần cẩn thận là dùng kiểu số đủ lớn trước khi lấy dư.

Greedy Gift Givers (gift1). Ghi tên từng người, sau đó duy trì hai mảng thu và chi. Với mỗi người tặng quà, chia đều số tiền cho n người nhận, cộng phần nhận vào từng người và giữ lại phần dư cho người tặng. Cuối cùng in $thu_i - chi_i$ theo đúng thứ tự đầu vào. Tìm tên tuyến tính cho từng giao dịch cho độ phức tạp xấp xỉ $O(n^3)$; dùng bảng băm ánh xạ tên sang chỉ số sẽ giảm còn $O(n^2)$ hoặc tốt hơn tùy cách cài đặt.

Friday the Thirteenth (friday). Mô phỏng theo tháng. Vì ngày 1/1/1900 là thứ Hai nên ngày 13/1/1900 là thứ Bảy; sau đó ngày 13 của tháng kế tiếp lệch thêm đúng số ngày của tháng hiện tại modulo 7. Mảng số ngày trong tháng phải điều chỉnh tháng Hai khi năm nhuận. Dữ liệu nhỏ nên mô phỏng từng tháng là đủ. Với dữ liệu lớn hơn có thể cộng theo năm: năm thường lệch 1 ngày, năm nhuận lệch 2 ngày sau tháng Hai. Chú ý thứ tự in của USACO và ký tự xuống dòng cuối.

Broken Necklace (beads). Cách trực tiếp thử mọi điểm cắt là $O(n^2)$. Có thể nhân đôi chuỗi để biến vòng thành đoạn thẳng rồi tiền xử lý bốn mảng: số hạt xanh/đỏ có thể gom sang trái và sang phải tại mỗi vị trí, trong đó hạt trắng được tính cho cả hai màu. Với mỗi điểm cắt, đáp án là

$$\min(n, \max(B_L(i), R_L(i)) + \max(B_R(i+1), R_R(i+1))).$$

Một cách khác cũng $O(n)$ là quét trên chuỗi nhân đôi, duy trì độ dài hai đoạn màu liên tiếp có thể gom; khi màu đổi thì dời đoạn sau thành đoạn trước. Luôn chặn kết quả bởi n .

Section 1.2

Milking Cows (milk2). Sắp các khoảng theo thời điểm bắt đầu rồi quét từ trái sang phải. Giữ khoảng hiện tại $[l, r]$; nếu khoảng kế tiếp bắt đầu không muộn hơn r , gộp và cập nhật r . Nếu nó bắt đầu sau r , cập nhật thời gian vắt sữa liên tục dài nhất bằng $r - l$, thời gian nghỉ dài nhất bằng khoảng hở, rồi mở khoảng mới. Độ phức tạp $O(n \log n)$. Cũng có thể dùng mảng đánh dấu trên miền thời gian hoặc cây đoạn; với miền 10^6 , mảng boolean đủ đơn giản nhưng phải xử lý biên nửa mở của khoảng.

Transformations (transform). Mô phỏng đủ các phép biến đổi trên ma trận $N \times N$. Quay phải 90° : phần tử (i, j) đi tới $(j, n - i + 1)$. Phản chiếu ngang: (i, j) đi tới $(i, n - j + 1)$. Kiểm tra lần lượt các phép trong thứ tự yêu cầu của đề, gồm quay 90° , 180° , 270° , phản chiếu, phản chiếu rồi quay, không đổi và không hợp lệ. Viết hàm so sánh ma trận rõ ràng để tránh nhầm thứ tự.

Name That Number (namenum). Không nên sinh toàn bộ 3^{12} chuỗi từ số rồi tra từ điển. Cách tốt hơn là đọc từng tên trong từ điển, đổi từng chữ cái sang chữ số duy nhất của bàn phím, rồi so sánh với chuỗi đầu vào. Như vậy thời gian khoảng $O(5000 \cdot 12)$, bộ nhớ hằng số. Cần bỏ qua các chữ cái không xuất hiện trên bàn phím đề bài, đặc biệt là Q và Z; cũng nên kiểm tra độ dài trước để cắt sớm. Nếu không có tên phù hợp thì in NONE.

Palindromic Squares (palsquare). Duyệt 1 đến 300, bình phương từng số, đổi cả số ban đầu và bình phương sang cơ số B , rồi kiểm tra bình phương có phải palindrome. Khi đổi cơ số, các chữ số trên 9 dùng A, B, ...; với cơ số 20, chữ số 19 là J, không được dùng ở F. Cài đặt bằng phép chia lấy dư rồi đảo ngược chuỗi.

Dual Palindromes (dualpal). Từ $S + 1$ trở đi, kiểm tra từng số trong các cơ số 2 đến 10. Nếu một số là palindrome ở ít nhất hai cơ số thì đưa vào đáp án; dừng khi đã có N số. Dữ liệu nhỏ nên cách duyệt trực tiếp đủ nhanh.

Section 1.3

Mixing Milk (milk). Tham lam mua sữa theo giá tăng dần. Sắp tất cả nông dân theo giá, rồi lấy nhiều nhất có thể từ người rẻ nhất cho đến khi đủ lượng cần mua. Chứng minh trao đổi: nếu một nghiệm tối ưu mua sữa đắt trong khi còn sữa rẻ chưa mua, thay phần đắt bằng phần rẻ sẽ không tệ hơn. Vì giá nằm trong khoảng nhỏ, có thể gộp lượng sữa cùng giá bằng đếm tần suất và quét giá từ thấp lên, tránh sắp xếp.

Prime Cryptarithm (crypt1). Tìm tất cả phép nhân dạng ba chữ số nhân hai chữ số sao cho mọi chữ số xuất hiện đều thuộc tập cho trước, hai tích riêng là số ba chữ số và tích cuối là số bốn chữ số. Duyệt năm chữ số của thừa số, hoặc duyệt hai thừa số tạo được từ tập chữ số. Cắt nhánh sớm khi tích riêng vượt 999 hoặc chứa chữ số cấm. Vì số chữ số cho phép nhỏ, vét cạn có kiểm tra hợp lệ là đủ.

Barn Repair (barn1). Muốn tổng chiều dài ván nhỏ nhất thì tương đương để lại tổng khoảng trống không che lớn nhất. Sắp các chuồng có bò, ban đầu một tấm che từ chuồng bò đầu đến chuồng bò cuối. Các khoảng trống giữa hai chuồng có bò liên tiếp là ứng viên để cắt; chọn $M - 1$ khoảng trống lớn nhất không che. Nếu số tấm ván không ít hơn số chuồng có bò, đáp án là số chuồng có bò. Cũng có thể làm DP theo $f[i][j]$, độ dài ít nhất dùng i tấm để che đến con bò thứ j , nhưng tham lam là gọn nhất.

Calf Flac (calfflac). Bỏ qua ký tự không phải chữ cái và không phân biệt hoa thường khi so sánh, nhưng khi in phải giữ nguyên đoạn trong văn bản gốc. Có thể duyệt từng tâm palindrome, gồm cả tâm lẻ và tâm chẵn, mở rộng hai phía trên chuỗi đã lọc rồi ánh xạ về chỉ số gốc để in. Độ phức tạp $O(n^2)$ đủ cho giới hạn của bài. Các hướng nâng cao như DP hoặc suffix array là không cần thiết, nhưng phải cẩn thận với xuống dòng trong đầu vào.

Section 1.4

Packing Rectangles (packrec). Thử mọi hoán vị của bốn hình chữ nhật và mọi lựa chọn xoay từng hình. Với mỗi cấu hình, mô phỏng các sơ đồ ghép cơ bản trong đề; thực tế có sáu hình vẽ nhưng hai hình là tương đương về công thức. Với mỗi cách ghép, tính khung bao, cập nhật diện tích nhỏ nhất và lưu các cặp cạnh đã chuẩn hóa ($\min(w, h), \max(w, h)$). Cuối cùng sắp và in các kích thước không trùng.

The Clocks (clocks). Mỗi thao tác có thể dùng 0 đến 3 lần vì bốn lần quay trở lại trạng thái cũ, và thứ tự thao tác không ảnh hưởng. Do đó chỉ cần duyệt $4^9 = 262144$ tổ hợp, chọn tổ hợp có chuỗi thao tác từ điển nhỏ nhất làm đáp án. Có thể biểu diễn mỗi đồng hồ bằng trạng thái 0, 1, 2, 3 tương ứng 12, 3, 6, 9 giờ. Cài đặt bằng mảng nhỏ là đủ; bitmask ba bit cho mỗi đồng hồ giúp tăng tốc, nhưng cần nhớ trạng thái đích là tất cả bằng 0.

Arithmetic Progressions (ariprog). Tiền xử lý tất cả số dạng $p^2 + q^2$ với $0 \leq p, q \leq M$, đánh dấu bằng mảng boolean và lưu danh sách tăng dần. Duyệt điểm đầu a và công sai b ; kiểm tra N phần tử bằng mảng đánh dấu. Cắt nhánh quan trọng: nếu $a + (N - 1)b$ vượt giới hạn lớn nhất $2M^2$, không cần thử công sai lớn hơn cho cùng a . Kết quả phải in theo công sai tăng dần rồi điểm đầu tăng dần; có thể sắp kết quả hoặc duyệt theo công sai.

Mother's Milk (milk3). Tổng lượng sữa không đổi, nên trạng thái chỉ cần (a, b) , lượng trong thùng C suy ra được. Từ mỗi trạng thái có sáu phép rót giữa hai thùng; dùng DFS hoặc BFS với mảng $\text{vis}[a][b]$. Mỗi khi $a = 0$, ghi nhận lượng c có thể xuất hiện. Sau khi duyệt xong, in tất cả lượng c đã đánh dấu theo thứ tự tăng dần. Khi in danh sách, tránh dư khoảng trắng cuối.

Section 1.5

Number Triangles (numtri). DP tam giác cổ điển. Gọi $f[i][j]$ là tổng lớn nhất từ ô (i, j) xuống đáy. Khi tính từ dưới lên:

$$f[i][j] = a[i][j] + \max(f[i + 1][j], f[i + 1][j + 1]).$$

Vì mỗi hàng chỉ phụ thuộc hàng dưới, có thể dùng mảng một chiều. Thời gian $O(n^2)$, bộ nhớ $O(n)$.

Prime Palindromes (pprime). Không sàng toàn bộ đến 10^8 bằng mảng lớn. Sinh palindrome theo độ dài rồi kiểm tra nguyên tố bằng chia thử đến căn bậc hai. Tối ưu quan trọng: mọi palindrome có độ dài chẵn đều chia hết cho 11, ngoại trừ chính 11; số chẵn và số tận cùng bằng 5 cũng không thể là nguyên tố, trừ 2 và 5. Sinh nửa đầu rồi phản chiếu sẽ tạo các số theo thứ tự; chỉ in các số trong $[A, B]$.

SuperPrime Rib (sprime). DFS theo chữ số. Chữ số đầu chỉ có thể là 2, 3, 5, 7; các chữ số sau chỉ cần thử 1, 3, 7, 9. Mỗi lần nối thêm một chữ số, kiểm tra tiền tố mới có nguyên tố không; nếu có thì đi tiếp. Khi đạt đủ N chữ số thì in. Không cần tiền xử lý lớn, chỉ cần kiểm tra chia thử đến căn.

Checker Challenge (checker). Đặt hậu từng hàng bằng DFS, đánh dấu cột và hai đường chéo đã bị chiếm. Để đếm số nghiệm nhanh hơn, khai thác đối xứng: với n chẵn, chỉ thử nửa đầu của hàng đầu rồi nhân đôi số nghiệm; với n lẻ cần xử lý riêng cột giữa. Cũng có thể dùng bitmask cho cột và chéo để giảm hằng số. Phần in ba nghiệm đầu nên dùng DFS theo thứ tự cột tăng dần; phần đếm tổng có thể dùng phiên bản tối ưu riêng.

Chương 2

Section 2.1

The Castle (castle). Mỗi ô chứa bit tường: tây 1, bắc 2, đông 4, nam 8. Flood fill các phòng, gán màu thành phần và đếm diện tích. Sau đó duyệt các bức tường có thể phá; nếu hai phía thuộc hai phòng khác nhau, diện tích mới là tổng hai phòng. Để đúng quy tắc ưu tiên của đề, duyệt cột từ trái sang phải, trong mỗi cột duyệt hàng từ dưới lên, xét tường bắc trước rồi tường đông. DSU cũng dùng được, nhưng flood fill trực tiếp dễ kiểm soát thứ tự hơn.

Ordered Fractions (frac1). Duyệt mọi phân số a/b với $0 \leq a \leq b \leq N$, giữ lại phân số tối giản $\gcd(a, b) = 1$, rồi sắp bằng so sánh chéo $a_1b_2 < a_2b_1$. Đừng quên 0/1 và 1/1. Một cách thanh lịch hơn là sinh cây Stern–Brocot: giữa hai phân số a/b và c/d , mediant $(a + c)/(b + d)$ được sinh nếu mẫu không vượt N , cho ra thứ tự tăng sẵn.

Sorting A Three-Valued Sequence (sort3). Đếm số lượng 1, 2, 3 để biết ba đoạn đúng trong mảng đã sắp. Gọi $d[i][j]$ là số phần tử j đang nằm trong đoạn phải chứa i . Trước hết hoán đổi trực tiếp các cặp sai đối xứng $d[i][j]$ và $d[j][i]$. Phần sai còn lại chỉ là chu trình ba giá trị; mỗi chu trình cần hai phép đổi. Công thức tương đương thường dùng là đưa tất cả 1 về đầu rồi xử lý đoạn 2, 3; cần nhớ chu trình ba nhân 2, không phải 3.

Healthy Holsteins (holstein). Mỗi loại thức ăn có hai lựa chọn: dùng hoặc không. Vì số loại không quá 15, duyệt bitmask từ 1 đến $2^G - 1$, cộng dinh dưỡng và kiểm tra đạt yêu cầu. Chọn nghiệm có ít loại nhất; khi hòa, bitmask duyệt theo thứ tự tăng chỉ đúng nếu thứ tự từ điển được bảo đảm, nên an toàn hơn là so sánh danh sách chỉ số. DFS chọn/không chọn cũng được và có thể cắt khi số loại đã chọn không thể tốt hơn đáp án hiện tại.

Hamming Codes (hamming). Sinh mã theo thứ tự tăng. Với mỗi số ứng viên, tính khoảng cách Hamming tới tất cả mã đã chọn bằng số bit 1 trong phép xor; nếu mọi khoảng cách ít nhất D thì nhận. Số 0 luôn là lựa chọn đầu. Có thể tiền xử lý popcount cho các giá trị dưới 2^B hoặc dùng thao tác bit trực tiếp. Khi đã chọn đủ N mã, in mỗi dòng tối đa mười số.

Section 2.2

Preface Numbering (preface). Cách đơn giản là lập bảng biểu diễn La Mã cho từng chữ số ở hàng đơn vị, chục, trăm, nghìn, rồi chuyển tất cả số $1..N$ và cộng tần suất chữ cái. Với $N \leq 3500$, cách này đủ nhanh. Có thể thống kê theo từng vị trí thập phân để tối ưu: số lần xuất hiện của mẫu I, V, X ở hàng đơn vị lặp lại trong mỗi khối 10, còn hàng chục và trăm chỉ là cùng mẫu dời sang X, L, C và C, D, M . Chỉ in các ký hiệu có tần suất dương theo thứ tự chuẩn.

Subset Sums (subset). Tổng $1 + \dots + n = n(n+1)/2$. Nếu tổng lẻ thì không có cách chia. Nếu tổng chẵn, đặt $T = \frac{n(n+1)}{4}$ và làm DP số cách chọn tập con có tổng T :

$$f[j] += f[j-i] \quad \text{khi xét số } i.$$

Kết quả phải chia 2 vì mỗi cách chia hai tập được đếm hai lần. Một biến thể tránh chia đôi là chỉ xét $1..n-1$ và buộc n thuộc tập còn lại.

Runaround Numbers (runround). Duyệt các số lớn hơn M và kiểm tra trực tiếp: chữ số phải khác 0, không lặp, bắt đầu ở vị trí đầu, mỗi bước nhảy theo giá trị chữ số modulo độ dài. Sau đúng số chữ số, phải quay lại vị trí đầu và đã thăm mọi vị trí đúng một lần. Vì chữ số tối đa 9 và không lặp, kiểm tra là hằng số. Cũng có thể DFS sinh các số có chữ số khác nhau rồi chọn số nhỏ nhất lớn hơn M .

Party Lamps (lamps). Bốn nút bấm là các phép đảo bit; bấm cùng nút hai lần triệt tiêu. Vì vậy chỉ cần duyệt 2^4 tổ hợp cơ bản và giữ tổ hợp có số nút bấm x thỏa $x \leq C$ và $(C-x)$ chẵn. Trạng thái lặp chu kỳ 6 theo chỉ số bóng, nên với $N > 6$ chỉ cần xét sáu bóng đầu rồi lặp khi in. Có thể mã hóa sáu bit: nút 1 xor 111111_2 , nút 2 xor các vị trí lẻ, nút 3 xor các vị trí chẵn, nút 4 xor vị trí $1 \bmod 3$. Sau khi lọc ràng buộc bật/tắt, sắp kết quả tăng từ điển.

Section 2.3

The Longest Prefix (prefix). Gọi $f[i]$ cho biết tiền tố độ dài i của chuỗi chính có thể tạo được. Khởi tạo $f[0] = \text{true}$. Với mỗi i khả đạt, thử nối từng primitive p ; nếu chuỗi chính từ $i+1$ khớp p , đặt $f[i+|p|] = \text{true}$. Độ dài primitive tối đa 10, nên so khớp trực tiếp đủ nhanh. Có thể dùng hash hoặc KMP để kiểm tra khớp, nhưng mấu chốt là chỉ mở rộng từ các vị trí đã khả đạt và duy trì đáp án lớn nhất.

Cow Pedigrees (nocows). Đếm cây nhị phân đầy đủ có N nút và chiều cao đúng K , modulo 9901. Cách gọn là để $dp[n][h]$ là số cây dùng n nút có chiều cao không quá h . Khi n lẻ:

$$dp[n][h] = \sum_{\substack{l+r=n-1 \\ l, r \text{ lẻ}}} dp[l][h-1] dp[r][h-1].$$

Biên $dp[1][h] = 1$. Đáp án là $(dp[N][K] - dp[N][K-1] + 9901) \bmod 9901$. Cách khác đếm chiều cao đúng bằng cách yêu cầu ít nhất một cây con có chiều cao $K-1$; dùng mảng tích lũy cho cây thấp hơn để tránh đếm thiếu.

Zero Sum (zerosum). DFS điền toán tử giữa $1, 2, \dots, N$ theo thứ tự khoảng trắng, $+$, $-$ để tự nhiên sinh đúng thứ tự từ điển. Duy trì tổng hiện tại và hạng tử cuối: nối khoảng trắng thì thay hạng tử cuối x bằng $10x \pm$ số mới theo dấu của x ; cộng hoặc trừ thì thêm hạng tử mới. Vì độ sâu tối đa 8, cũng có thể sinh toàn bộ biểu thức rồi tính cuối cùng.

Money Systems (money). Đây là bài đếm số cách đổi tiền không giới hạn. Mảng một chiều $f[j]$ là số cách tạo giá trị j , khởi tạo $f[0] = 1$. Với từng mệnh giá c , quét j tăng từ c đến N và cộng $f[j-c]$ vào $f[j]$. Thứ tự quét mệnh giá bên ngoài bảo đảm các hoán vị khác nhau của cùng tập tiền không bị đếm lặp. Dùng kiểu số 64 bit.

Controlling Companies (concom). Duy trì ma trận phần trăm sở hữu trực tiếp và tổng sở hữu mà công ty i có được khi đã kiểm soát các công ty khác. Với từng công ty nguồn i , bắt đầu từ sở hữu trực tiếp, mỗi khi tổng sở hữu ở công ty j vượt 50% và j chưa được mở rộng, cộng toàn bộ cổ phần của j vào tổng của i . Lặp đến khi không có công ty mới. In mọi cặp i, j với $i \neq j$ được kiểm soát.

Section 2.4

The Tamworth Two (ttwo). Mô phỏng trạng thái của nông dân và bò trên lưới 10×10 . Mỗi đối tượng có vị trí và hướng; nếu ô phía trước hợp lệ và không có vật cản thì đi tới, ngược lại quay phải. Nếu hai vị trí trùng nhau thì in thời gian. Không gian trạng thái hữu hạn $10^2 \cdot 4 \cdot 10^2 \cdot 4$, nên nếu sau một số bước trạng thái lặp lại mà chưa gặp nhau thì đáp án là 0. Cài đặt thường chỉ cần mô phỏng tối đa 160000 bước.

Overfencing (maze1). Chuyển mê cung ASCII thành đồ thị các ô. Tìm hai lối ra trên biên, rồi BFS đồng thời từ cả hai lối ra để lấy khoảng cách ngắn nhất từ mỗi ô tới lối ra gần nhất. Đáp án là khoảng cách lớn nhất. Có thể biểu diễn mê cung bằng mảng kích thước $(2H + 1) \times (2W + 1)$, trong đó đường thông giữa hai ô được xác định bởi ký tự khoảng trắng. BFS nhanh và đơn giản hơn flood fill đệ quy.

Cow Tours (cowntour). Dùng Floyd–Warshall tính khoảng cách ngắn nhất trong từng thành phần liên thông. Với mỗi điểm i , tính $far[i]$, khoảng cách xa nhất từ i tới điểm cùng thành phần. Với mỗi thành phần, tính đường kính hiện có $dia[c]$. Khi nối hai điểm chưa liên thông i, j , đường kính mới là

$$\max(dia[c_i], dia[c_j], far[i] + \text{dist}(i, j) + far[j]).$$

Lấy giá trị nhỏ nhất trên mọi cặp i, j khác thành phần. Cảnh báo quan trọng: không chỉ tối thiểu hóa $far[i] + d + far[j]$; phải so với đường kính cũ của cả hai thành phần.

Bessie Come Home (comehome). Đồ thị chỉ có 52 đỉnh là chữ cái hoa và thường. Đọc cạnh, nếu có nhiều cạnh giữa cùng hai đỉnh thì giữ trọng số nhỏ nhất. Có thể dùng Floyd–Warshall rồi tìm chữ cái hoa từ A đến Y gần Z nhất. Dijkstra từ Z cũng được và nhanh hơn, nhưng với kích thước này Floyd dễ viết.

Fractions to Decimals (fracdec). In phần nguyên trước. Với phần dư, mô phỏng phép chia dài: mỗi lần nhân dư với 10, lấy chữ số thương mới và dư mới. Nếu dư bằng 0, số thập phân kết thúc. Nếu một dư xuất hiện lại, các chữ số từ lần xuất hiện đầu của dư đó đến hiện tại là chu kỳ và phải đặt trong ngoặc. Lưu vị trí đầu tiên của từng dư trong mảng kích thước mẫu số. Khi in, xuống dòng sau mỗi 76 ký tự theo yêu cầu USACO.

Chương 3

Section 3.1

Agri-Net (agrinet). Đây là bài cây khung nhỏ nhất chuẩn trên đồ thị đầy đủ cho bởi ma trận trọng số. Prim $O(n^2)$ rất phù hợp: giữ khoảng cách nhỏ nhất từ mỗi đỉnh chưa chọn tới cây hiện tại, mỗi bước lấy đỉnh chưa chọn có khoảng cách nhỏ nhất và cập nhật. Kruskal cũng được nhưng phải sinh nhiều cạnh hơn.

Score Inflation (inflate). Bài ba lô hoàn toàn. Mỗi loại bài có điểm p_i và thời gian t_i , có thể làm nhiều lần trong tổng thời gian M . Mảng $f[j]$ là điểm tối đa trong j phút; với từng bài, quét j tăng từ t_i đến M :

$$f[j] = \max(f[j], f[j - t_i] + p_i).$$

Kết quả là $f[M]$.

Humble Numbers (humble). Đặt $hum[0] = 1$. Với mỗi prime p_i , giữ con trỏ idx_i sao cho $hum[idx_i]p_i$ là ứng viên nhỏ nhất lớn hơn humble vừa sinh. Mỗi bước lấy min các ứng viên làm humble kế tiếp; với mọi prime tạo ra đúng giá trị đó, tăng con trỏ để loại trùng. Đây là cách $O(NK)$, ổn định hơn dùng priority queue vì tránh bùng bộ nhớ.

Shaping Regions (rect1). Mô phỏng tô hình chữ nhật theo thứ tự. Cách thường dùng là duy trì danh sách các mảnh hình chữ nhật còn nhìn thấy; khi một hình mới phủ lên, cắt các mảnh cũ giao với hình mới thành những phần không bị phủ, rồi thêm hình mới với màu của nó. Sau cùng cộng diện tích theo màu. Có thể rời rạc hóa tọa độ rồi quét, nhưng danh sách mảnh để giữ bộ nhớ thấp hơn nếu cắt cẩn thận.

Contact (contact). Vì $A \leq B \leq 12$, mã hóa mỗi chuỗi nhị phân bằng giá trị số và độ dài. Khi quét tín hiệu, với mỗi vị trí sinh tất cả độ dài $A..B$ còn nằm trong chuỗi, tăng tần suất. Để phân biệt 0 và 00, hoặc lưu thêm độ dài, hoặc thêm bit dẫn 1 khi mã hóa. Khi in, sắp theo tần suất giảm dần, độ dài tăng dần, giá trị nhị phân tăng dần; mỗi dòng tối đa sáu mẫu.

Stamps (stamps). Gọi $f[x]$ là số tem ít nhất để tạo bưu phí x . Khởi tạo $f[0] = 0$, các giá trị khác là vô cực. Tăng x từ 1 lên, tính

$$f[x] = 1 + \min_{v_i \leq x} f[x - v_i].$$

Số đầu tiên có $f[x] > K$ hoặc không tạo được cho biết đáp án là $x - 1$. Chỉ cần tính tới khi gặp khoảng hở đầu tiên; nếu mọi giá trị liên tiếp đến đó tạo được bằng không quá K tem thì các giá trị trước đó hợp lệ.

Section 3.2

Factorials (fact4). Muốn chữ số cuối khác 0 của $N!$, phải loại các cặp $2 \cdot 5$. Trong phân tích thừa số, số 2 nhiều hơn số 5, nên khi nhân từng i , loại hết nhân tử 5 và ghi số lượng, đồng thời loại tương ứng số nhân tử 2. Cuối cùng nhân lại các 2 dư và lấy modulo 10. Một cách thực dụng khác là duy trì vài chữ số cuối khác 0, sau mỗi phép nhân xóa các số 0 cuối rồi lấy modulo đủ lớn để tránh tràn.

Stringsobits (kimbits). Tiền xử lý $cnt[n][l]$, số chuỗi nhị phân dài n có nhiều nhất l bit 1:

$$cnt[n][l] = cnt[n - 1][l] + cnt[n - 1][l - 1],$$

với $cnt[0][l] = 1$ và $cnt[n][0] = 1$. Để dựng chuỗi thứ I , xét từ bit cao xuống: số chuỗi bắt đầu bằng 0 là $cnt[pos - 1][L]$. Nếu I không vượt số này, in 0; ngược lại in 1, trừ số đó khỏi I và giảm L . Dùng số nguyên 64 bit là đủ cho giới hạn 31 bit.

Spinning Wheels (spin). Sau 360 giây, mọi bánh xe trở lại vị trí ban đầu, nên mô phỏng $t = 0..359$. Với mỗi thời điểm, đánh dấu các góc được khe của từng bánh che phủ, có xét wrap-around modulo 360. Nếu tồn tại góc được cả năm bánh mở thì in thời điểm đó. Chú ý biên đoạn: dữ liệu USACO coi các đầu mút của khe là có thể truyền sáng, nên dùng đoạn đóng theo độ dài đề cho.

Feed Ratios (ratios). Cần tìm x, y, z, k sao cho tổ hợp ba loại thức ăn bằng k lần vector mục tiêu. Dữ liệu nhỏ nên duyệt x, y, z trong $0..99$, tính vector tổng, rồi kiểm tra có là bội dương của mục tiêu không; chọn tổng $x + y + z$ nhỏ nhất theo yêu cầu. Cách toán học dùng định thức hoặc khử Gauss trên hệ tuyến tính, nhưng phải loại nghiệm âm, nghiệm phân số và trường hợp mục tiêu có thành phần 0.

Magic Squares (msquare). BFS trên $8!$ hoán vị từ trạng thái 12345678. Ba phép biến đổi A, B, C sinh trạng thái mới; khi gặp đích thì đường đi BFS là ngắn nhất, và nếu mở rộng theo thứ tự A, B, C thì cũng nhỏ từ điển. Dùng Cantor expansion, mảng bảy chiều, hoặc mã hóa octal/bit để đánh dấu đã thăm. Nhớ xử lý trường hợp trạng thái đầu đã là đích.

Sweet Butter (butter). Mô hình: chọn một đồng cỏ làm điểm hợp sao cho tổng khoảng cách ngắn nhất từ các con bò đến đó nhỏ nhất. Đồ thị thưa, cần lưu bằng danh sách kề. Chạy Dijkstra có heap từ từng đồng cỏ ứng viên, hoặc SPFA nếu cài tốt; sau mỗi lần cộng khoảng cách tới các vị trí có bò. Floyd $O(P^3)$ quá chậm với $P \leq 800$. Nếu có nhiều bò cùng đồng cỏ, nhân khoảng cách với số bò ở đó.

Section 3.3

Riding The Fences (fence). Tìm đường Euler trong đồ thị vô hướng đa cạnh. Nếu có đỉnh bậc lẻ, bắt đầu từ đỉnh lẻ nhỏ nhất; nếu không, bắt đầu từ đỉnh nhỏ nhất có cạnh. Dùng thuật toán Hierholzer: luôn đi cạnh nhỏ nhất khả dụng, xóa cạnh khi đi, khi không còn cạnh thì thêm đỉnh vào kết quả. Kết quả thu được ngược thứ tự nên in đảo lại. Vì có đa cạnh, mà trộn đếm cạnh giữa hai đỉnh là tiện.

Shopping Offers (shopping). Chỉ có tối đa 5 mặt hàng cần mua, mỗi số lượng không quá 5, nên dùng DP năm chiều hoặc mã hóa cơ số 6. Trạng thái là số lượng đã mua của từng mặt hàng, $f[state]$ là chi phí nhỏ nhất. Các offer đặc biệt và việc mua đơn lẻ đều là các chuyển tiếp; chỉ áp dụng khi không vượt nhu cầu. Khởi tạo $f[0] = 0$, duyệt toàn bộ trạng thái để cập nhật.

Camelot (camelot). Trước hết BFS nước đi mã từ mọi ô để có khoảng cách giữa mọi cặp ô. Với mỗi ô hợp, cộng khoảng cách của tất cả kỵ sĩ tới ô đó. Nhà vua có thể tự đi bằng khoảng cách Chebyshev, hoặc một kỵ sĩ đón vua tại một ô gần vua rồi cùng tới ô hợp. Vì vua chậm, chỉ cần thử điểm đón trong một vùng nhỏ quanh vua, ví dụ lệch không quá 2 ô theo mỗi chiều. Với mỗi kỵ sĩ, tính phần tiết kiệm/chi phí thêm

$$d(knight, pick) + d(pick, meet) + king(pick) - d(knight, meet).$$

Lấy chi phí thêm nhỏ nhất cộng vào tổng của ô hợp. Cắt nhánh khi tổng hiện tại đã vượt đáp án.

Home on the Range (range). DP hình vuông toàn 1. Gọi $dp[i][j]$ là cạnh lớn nhất của hình vuông có góc dưới phải tại (i, j) . Nếu ô là 1:

$$dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]).$$

Mỗi giá trị $dp[i][j] = s$ đóng góp một hình vuông cho mọi cạnh $2..s$. Cộng dồn tần suất rồi in các cạnh có số lượng dương. Có thể định nghĩa góc trên trái tương tự; công thức chỉ đổi hướng.

A Game (game1). DP trò chơi hai người tối ưu. Gọi $sum[i][j]$ là tổng đoạn và $f[i][j]$ là điểm tối đa người đi trước trong đoạn $[i, j]$ có thể lấy. Khi lấy trái hoặc phải, đối thủ trở thành người đi trước trên đoạn còn lại:

$$f[i][j] = \max(a_i + sum[i+1][j] - f[i+1][j], a_j + sum[i][j-1] - f[i][j-1]).$$

Biên $f[i][i] = a_i$. Điểm người thứ nhất là $f[1][n]$, người thứ hai là $sum[1][n] - f[1][n]$.

Section 3.4

Closed Fences (fence4). Trước tiên kiểm tra đa giác hợp lệ bằng cách bảo đảm không có hai cạnh không kề nhau cắt nhau thật sự. Để xác định cạnh nhìn thấy, bắn tia từ điểm quan sát tới các đỉnh và điểm giữa cạnh; cạnh giao gần nhất với tia là cạnh có thể nhìn thấy. Các trường hợp suy biến rất quan trọng: tia đi đúng qua đỉnh, qua cạnh song song, hoặc chỉ chạm một cạnh không tạo vùng nhìn thấy. Một cách xử lý là phân biệt giao cắt xuyên qua tia và giao tại đỉnh, giữ khoảng cách gần nhất ở hai phía trái/phải của tia để loại trường hợp bị che hoàn toàn.

American Heritage (heritage). Từ preorder và inorder dựng postorder bằng đệ quy. Ký tự đầu của preorder là gốc; tìm nó trong inorder để biết kích thước cây con trái và phải. Đệ quy trên hai đoạn tương ứng, in hậu tự là hậu tự trái, hậu tự phải, rồi gốc. Không nhất thiết dựng cấu trúc cây thật nếu chỉ cần in postorder.

Electric Fence (fence9). Dùng định lý Pick: với đa giác lưới, $S = I + B/2 - 1$, trong đó I là số điểm lưới bên trong và B là số điểm lưới trên biên. Tam giác có các đỉnh $(0,0)$, (N,M) , $(P,0)$, diện tích $S = PM/2$. Số điểm lưới trên đoạn giữa hai điểm có độ lệch (dx, dy) là $\gcd(|dx|, |dy|) + 1$; khi cộng ba cạnh, dùng $B = \gcd(N, M) + \gcd(|P - N|, M) + P$. Thay vào Pick để lấy I .

Raucous Rockers (rockers). Thứ tự bài hát phải được giữ cả trong từng đĩa và giữa các đĩa. Dữ liệu nhỏ nên có thể duyệt chọn/không chọn, nhưng DP rõ hơn: $f[i][j][r]$ là số bài tối đa xét từ bài i , còn j đĩa và đĩa hiện tại còn r phút. Chuyển bằng bỏ bài, đặt bài vào đĩa hiện tại nếu đủ chỗ, hoặc mở đĩa mới nếu còn. Một biến thể tính $best[l][r]$, số bài nhiều nhất trong đoạn cho một đĩa bằng ba lô 0/1, rồi DP theo số đĩa.

Chương 4

Section 4.1

Beef McNuggets (nuggets). Nếu gcd của các kích thước gói không phải 1, có vô hạn số lượng không biểu diễn được nên in 0. Ngược lại, dùng DP khả đạt trên các tổng tăng dần. Khi đã gặp một đoạn liên tiếp dài bằng kích thước gói nhỏ nhất mà mọi giá trị đều đạt được, mọi số lớn hơn cũng đạt được; số không đạt lớn nhất trước đó là đáp án. Giới hạn thực tế cũng có thể chặn bởi tích hoặc bội chung của hai kích thước lớn, nhưng tiêu chí đoạn liên tiếp dễ cài hơn.

Fence Rails (fence8). Bản chất là tìm số rail nhiều nhất có thể cắt từ các board. Sắp rail tăng dần, board tăng dần; nhị phân hoặc tăng dần số rail cần kiểm tra. Với một số k , chỉ cần dùng k rail ngắn nhất, nhưng trong DFS nên đặt các rail dài trước để sớm thất bại. Cắt nhánh quan trọng: nếu phần gỗ bỏ đi không thể dùng cho rail nhỏ nhất đã vượt tổng dư cho phép, dừng. Khi hai rail bằng nhau, ép rail sau dùng board có chỉ số không nhỏ hơn rail trước để tránh hoán vị trùng; tương tự có thể xử lý board bằng nhau.

Fence Loops (fence6). Đầu vào mô tả quan hệ giữa các cạnh, không phải đỉnh. Cần khôi phục các đỉnh: mỗi đoạn fence có hai đầu, các đoạn được liệt kê ở cùng một phía phải thuộc cùng đỉnh; có thể

dùng DSU để gộp các đầu đoạn tương ứng. Sau khi dựng đồ thị trọng số, tìm chu trình nhỏ nhất bằng cách với mỗi cạnh (u, v, w) , tạm bỏ cạnh đó và tìm đường ngắn nhất từ u đến v ; $dist(u, v) + w$ là một chu trình ứng viên. Lấy min trên mọi cạnh. Một biến thể dùng Floyd cải tiến để tìm chu trình nhỏ nhất.

Cryptcowgraphy (cryptcow). Tìm kiếm ngược từ chuỗi mã hóa về câu đích. Mỗi bước chọn một bộ $C, 0, W$, hoán đổi hai đoạn giữa chúng rồi xóa ba ký tự đó. Cắt nhánh quyết định: độ dài phải bằng độ dài đích cộng $3k$; tần suất mọi ký tự ngoài COW phải khớp; tiền tố và hậu tố không chứa COW phải khớp tiền tố/hậu tố đích; mọi đoạn giữa hai ký tự COW liên tiếp phải xuất hiện trong câu đích. Dùng hash các chuỗi đã xét để tránh lặp. Thứ tự thử tốt là duyệt vị trí 0 trước, rồi C và W.

Section 4.2

Drainage Ditches (ditch). Mô hình max flow từ nguồn 1 đến đích N . Đồ thị có thể có nhiều cạnh giữa cùng hai đỉnh, nên với ma trận dung lượng hãy cộng dồn dung lượng khi đọc cạnh; với danh sách cạnh thì thêm từng cạnh kèm cạnh ngược. Edmonds–Karp đã đủ cho giới hạn, Dinic hoặc preflow-push nhanh hơn. Các vòng không cần xử lý riêng vì bảo toàn luồng tự loại chúng khỏi nghiệm tối ưu.

The Perfect Stall (stall4). Đây là matching cực đại hai phía giữa bò và chuồng. Dùng thuật toán Kuhn: với từng bò, DFS tìm chuồng chưa ghép hoặc có thể đẩy bò đang ghép sang chuồng khác. Mỗi lần DFS cần mảng thăm riêng cho phía chuồng. Cũng có thể dựng mạng nguồn–bò–chuồng–đích với dung lượng 1, nhưng Hungarian/Kuhn ngắn và đủ nhanh.

Job Processing (job). Để tính thời điểm hoàn thành qua các máy loại A , luôn giao công việc tiếp theo cho máy có thời điểm hoàn thành sớm nhất $delay_i + time_i$, rồi tăng $delay_i$. Thu được dãy $A[1] \leq \dots \leq A[N]$. Làm tương tự cho máy loại B , thu dãy B . Thời gian hoàn thành sau công đoạn A là $A[N]$. Để tối thiểu hóa toàn bộ hai công đoạn, ghép công việc hoàn thành sớm ở A với thời gian xử lý muộn ở B : đáp án là $\max_i A[i] + B[N + 1 - i]$. Nếu số máy lớn, dùng heap để chọn máy sớm nhất.

Cowcycles (cowcycle). DFS chọn dãy bánh trước và bánh sau tăng dần trong các khoảng cho trước. Điều kiện max ratio ít nhất gấp ba min ratio có thể kiểm bằng nhân chéo để tránh số thực. Với mỗi cấu hình hợp lệ, sinh tất cả tỉ số, sắp tăng, lấy chênh lệch liên tiếp và tính phương sai; giữ cấu hình có phương sai nhỏ nhất, hòa thì do DFS tăng dần sẽ cho thứ tự mong muốn nếu cập nhật cẩn thận. Vì số lượng tỉ số nhỏ, insertion sort thường nhanh hơn quicksort.

Section 4.3

Buy Low, Buy Lower (buyLow). Tính longest decreasing subsequence và số dãy khác nhau. $len[i]$ là độ dài LDS kết thúc tại i , $cnt[i]$ là số dãy như vậy. Khi xét $j < i$ với $a[j] > a[i]$, cập nhật độ dài và cộng số cách nếu đạt cùng độ dài. Để tránh đếm trùng do giá trị bằng nhau, khi cộng các j cùng độ dài chỉ dùng một giá trị $a[j]$ một lần, thường bằng cách quét j từ phải sang trái và ghi giá trị đã cộng. Thêm giá trị 0 cuối dãy giúp gom đáp án về một trạng thái. Số cách cần big integer cộng.

The Primes (prime3). Phải điền lưới 5×5 sao cho mọi hàng, cột và hai đường chéo là số nguyên tố năm chữ số có tổng chữ số cho trước. Tiền xử lý các prime năm chữ số theo tổng chữ số và theo các mẫu chữ số đã biết. Thứ tự tìm kiếm rất quan trọng: điền hai đường chéo trước vì chúng ràng buộc nhiều ô, sau đó điền cột giữa, hàng trên, hàng dưới, rồi các hàng/cột còn lại; nhiều ô cuối có thể suy ra bằng tổng chữ số. Luôn cắt nhánh bằng bảng prime theo tiền tố/mẫu, cuối cùng xác nhận mọi hàng cột là prime.

Street Race (race3). Với mỗi đỉnh không phải nguồn và đích, tạm loại nó rồi DFS từ nguồn; nếu không tới được đích thì đó là unavoidable point. Để kiểm tra splitting point, với mỗi unavoidable point v , đánh dấu các đỉnh tới được từ nguồn khi loại v , rồi DFS từ v trên đồ thị gốc. Nếu hai tập này không giao nhau, v chia mọi đường đi thành hai phần độc lập và là splitting point. Cách tô màu đỏ/đen trong nguồn chính là phép kiểm tra hai tập này.

Letter Game (lgame). Đọc từ điển và loại ngay các từ không thể tạo từ multiset chữ cái đầu vào. Tính điểm từng từ hợp lệ. Đáp án có thể là một từ hoặc cặp hai từ; duyệt một từ và mọi cặp từ hợp lệ, kiểm tra tổng tần suất không vượt tần suất đầu vào, cập nhật điểm lớn nhất. Vì từ dài 3..7, chỉ cần tối đa hai từ. Lưu kết quả dưới dạng chuỗi một hoặc hai từ theo thứ tự từ điển trong cặp, rồi sắp toàn bộ kết quả trước khi in.

Section 4.4

Shuttle Puzzle (shuttle). Có công thức cấu tạo dãy vị trí ô trống. Nếu tính cả vị trí ban đầu, dãy chia thành $2N + 1$ nhóm có độ dài $1, 2, \dots, N, N + 1, N, \dots, 1$. Trong nửa đầu, nhóm lẻ là cặp số cộng giảm công sai 2, nhóm chẵn là cặp số cộng tăng công sai 2; nửa sau đối xứng. In từ bước di chuyển đầu tiên, mỗi dòng tối đa hai mươi số. DFS cũng có thể làm nhưng không cần thiết và dễ chậm.

Pollutant Control (milk6). Dùng định lý max-flow min-cut. Cần tối thiểu hóa theo ba tiêu chí: tổng dung lượng cắt, số cạnh bị cắt, rồi thứ tự chỉ số cạnh. Mẹo chuẩn là biến dung lượng cạnh c thành $c \cdot B + 1$ với B lớn hơn số cạnh, hoặc dùng hệ số lớn hơn nữa để mã hóa cả chỉ số, sao cho max flow mới vẫn ưu tiên dung lượng gốc trước. Sau khi có max flow, tìm cạnh thuộc min cut bằng residual graph: các cạnh từ tập còn tới được từ nguồn sang tập không tới được tạo một lát cắt. Với cạnh song song, khi tính luồng có thể gộp dung lượng nhưng khi xuất phải quy về cạnh gốc cẩn thận.

Frame Up (frameup). Tìm khung chữ nhật của từng chữ cái bằng min/max hàng cột. Nếu trên biên của khung A xuất hiện chữ cái $B \neq A$, thì B nằm trên A , tức B phải được gỡ trước A ; thêm cạnh $B \rightarrow A$. Sau đó liệt kê mọi thứ tự topo của DAG và in theo thứ tự từ điển. Chữ cái có thể không liên tục, nên chỉ xét các chữ cái thật sự xuất hiện. Một cách khác là DFS thử gỡ các khung đang hoàn toàn lộ ra, nhưng topo rõ hơn.

Chương 5

Section 5.1

Fencing the Cows (fc). Dựng convex hull của tập điểm rồi tính chu vi. Graham scan: chọn điểm thấp nhất, trái nhất làm gốc, sắp các điểm còn lại theo góc cực bằng tích có hướng, rồi dùng stack; khi ba điểm cuối tạo rẽ phải hoặc không giữ lồi theo quy ước, pop đỉnh giữa. Mỗi điểm vào/ra stack tối đa một lần, tổng thời gian $O(n \log n)$ do bước sắp xếp. Sau hull, cộng khoảng cách Euclid giữa các đỉnh liên tiếp.

Starry Night (starry). Flood fill từng cụm sao. Với mỗi cụm, chuẩn hóa tọa độ tương đối trong bounding box rồi so với các cụm đã biết dưới tám phép biến đổi: bốn phép quay và phản chiếu. Có thể cắt nhanh bằng số ô sao và kích thước bounding box sau quay. Nếu khớp cụm cũ, dùng cùng chữ cái; nếu không, cập chữ cái mới. Hash dạng chuẩn nhỏ nhất của tám biến thể là cách gọn để so sánh.

Musical Themes (theme). Hai chủ đề giống nhau nếu hiệu giữa các nốt liên tiếp giống nhau, nên chuyển dãy nốt thành dãy sai phân. Bài toán trở thành tìm hai đoạn bằng nhau, không chồng lấn, có độ dài sai phân ít nhất 4 để chủ đề gốc dài ít nhất 5. DP $O(n^2)$: nếu $diff[i] = diff[j]$, đặt $lcp[i][j] = lcp[i+1][j+1] + 1$, nhưng chặn bởi $j - i$ để tránh chồng lấn. Cập nhật đáp án bằng $lcp + 1$ trên dãy nốt. Cũng có thể nhị phân độ dài và dùng hash hoặc suffix array.

Section 5.2

Snail Trail (snail). DFS từ góc trên trái. Ở mỗi bước, chọn hướng, rồi trượt thẳng đến khi gặp biên, vật cản hoặc ô đã đi. Đánh dấu tất cả ô trên đoạn vừa đi, cập nhật độ dài tốt nhất; nếu dừng vì ô đã đi thì nhánh kết thúc, còn nếu dừng vì biên/vật cản thì thử rẽ trái/phải. Khi quay lui, bỏ đánh dấu đúng các ô vừa thêm. Chú ý đọc cột dạng chữ cái; đề bảo đảm cột vật cản không vượt quá phạm vi biểu diễn.

Electric Fences (fence3). Cần tìm điểm tối thiểu hóa tổng khoảng cách tới các đoạn thẳng. Vì yêu cầu chính xác không quá gắt, có thể dùng tìm kiếm cục bộ hoặc mô phỏng annealing: bắt đầu tại một điểm, thử các hướng xung quanh với bước lớn, nhận điểm tốt hơn, rồi giảm bước dần đến khi đủ nhỏ. Khoảng cách từ điểm tới đoạn là khoảng cách tới hình chiếu nếu hình chiếu nằm trong đoạn, ngược lại min khoảng cách tới hai đầu. Một phương án nghiêm ngặt hơn là tam phân lồng nhau trên hàm lồi theo hai biến.

Wisconsin Squares (wissqu). Đây là bài tìm kiếm có dữ liệu rất nhỏ. DFS đặt lần lượt các ký tự theo ràng buộc đề bài, đánh dấu ô đã dùng và kiểm tra điều kiện lân cận trước khi đặt. Cần tối ưu hằng số: dùng mảng cố định, tránh cấp phát trong đệ quy và cắt nhánh sớm. Nguồn ghi chú cho biết bộ test chủ yếu là mẫu, nhưng vẫn nên viết tìm kiếm tổng quát thay vì hard-code.

Section 5.3

Milk Measuring (milk4). Yêu cầu ít thùng đo nhất, và trong cùng số lượng thì danh sách nhỏ nhất. Dùng iterative deepening: thử chọn $d = 1, 2, \dots$ thùng theo thứ tự tăng. Với mỗi tập thùng, kiểm tra có đo được Q không bằng DP unbounded: $can[0] = true, can[x] \mid= can[x - v]$. Vì mỗi thùng trong tập chọn nên hữu ích, có thể khởi tạo hoặc cắt nhánh để tránh tập có thùng không bao giờ dùng. Khi tập đầu tiên hợp lệ theo thứ tự duyệt, in ngay.

Window Area (window). Duy trì thứ tự chồng của các cửa sổ. Các thao tác tạo, đưa lên trên, đưa xuống dưới, xóa chỉ sửa danh sách. Với truy vấn diện tích nhìn thấy của cửa sổ W , bắt đầu từ chính hình chữ nhật của W , lần lượt lấy các cửa sổ nằm trên nó để cắt phần bị che. Khi một hình chữ nhật A bị hình B phủ, tách A thành tối đa bốn phần không giao nhau quanh giao $A \cap B$, giữ lại các phần còn diện tích dương. Tổng diện tích các mảnh còn lại chia diện tích ban đầu là phần trăm nhìn thấy. Tọa độ hai góc nhập vào có thể đảo thứ tự, cần chuẩn hóa min/max.

Network of Schools (schlnet). Co các strongly connected components thành DAG. Câu một là số SCC có indegree bằng 0, vì mỗi nguồn của DAG phải nhận phần mềm ban đầu. Câu hai là số cạnh ít nhất cần thêm để DAG trở thành strongly connected: nếu chỉ có một SCC thì 0, ngược lại là $\max(\#nguồn, \#đích)$, trong đó nguồn có indegree 0, đích có outdegree 0. Kosaraju hoặc Tarjan đều $O(n + m)$; Floyd reachability cũng qua được nhưng kém tổng quát.

Big Barn (bigbrn). Giống bài hình vuông lớn nhất không chứa chương ngại. Đánh dấu ô có cây. Nếu ô (i, j) trống, với $dp[i][j]$ là cạnh lớn nhất của hình vuông có góc dưới phải tại đó:

$$dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]).$$

Nếu có cây thì $dp = 0$. Lấy max toàn bảng. Có thể định nghĩa góc trên trái như nguồn ghi chú; kết quả tương đương. Bộ nhớ có thể dùng rolling rows.

Section 5.4

All Latin Squares (latin). Tìm kiếm là hướng chính. Cố định hàng đầu là $1..N$; do đối xứng hoán vị ký hiệu, các lựa chọn ở cột đầu có số nghiệm tương đương, nên có thể giảm mạnh không gian bằng cách chỉ tìm phần vuông $(N-1) \times (N-1)$. Dùng bitmask cho các số đã dùng trong từng hàng và cột. Các tối ưu sâu hơn dùng cấu trúc chu trình của hoán vị giữa hai hàng để gom các nhánh tương đương, nhưng cốt lõi vẫn là DFS có đối xứng.

Canada Tour (tour). Đảo chiều đường về, bài toán trở thành tìm hai đường đi không giao nhau từ thành phố 1 đến thành phố N trong DAG theo thứ tự tây–đông, sao cho tổng số thành phố khác nhau lớn nhất. $DP f[i][j]$ là số thành phố đã thăm khi hai người đang ở i và j . Chỉ mở rộng chỉ số lớn hơn để tránh đếm lại; khi có cạnh $k \rightarrow j$, cập nhật trạng thái. Đáp án là $\max f[i][N]$ với cạnh $i \rightarrow N$. Có thể mô hình bằng max-cost max-flow sau khi tách đỉnh dung lượng 1, riêng nguồn và đích dung lượng 2.

Character Recognition (charrec). Trích 27 ký tự mẫu từ font.in, mỗi ký tự chuẩn cao 20 dòng. Tiền xử lý $diff[ch][fontRow][inputRow]$, số bit khác nhau giữa một dòng mẫu và một dòng đầu vào. DP theo dòng: $dp[i]$ là sai khác nhỏ nhất khi đã nhận dạng đến dòng i . Một ký tự trong ảnh có thể cao 19, 20, 21 dòng do thiếu hoặc thừa một dòng. Với 20, cộng trực tiếp. Với 19, thử dòng mẫu bị thiếu; với 21, thử dòng ảnh bị thừa. Lưu ký tự chọn để truy vết chuỗi kết quả.

Betsy's Tour (betsy). DFS đếm đường Hamilton từ góc trên trái đến góc dưới trái. Tối ưu quan trọng là tránh tạo ngõ cụt: nếu quanh vị trí hiện tại có hơn một ô bắt buộc phải đi ngay, nhánh vô nghiệm; nếu có đúng một ô bắt buộc, chỉ đi ô đó. Cắt tiếp khi đường đi chia phần chưa thăm thành hai vùng, ví dụ trái/phải đã bị chặn còn trên/dưới đều mở, hoặc ngược lại. Với các cắt nhánh này, $N = 7$ chạy nhanh. Có thể dùng DP trạng thái liên thông, nhưng DFS được tối ưu là đủ.

TeleCowmunication (telecow). Bài min vertex cut. Tách mỗi đỉnh i thành $i_{in} \rightarrow i_{out}$ có dung lượng 1; mỗi cạnh vô hướng (i, j) trở thành hai cung $i_{out} \rightarrow j_{in}$ và $j_{out} \rightarrow i_{in}$ dung lượng vô cực. Max flow từ nguồn tới đích cho kích thước min cut. Để lấy danh sách đỉnh nhỏ nhất từ điển, duyệt đỉnh tăng dần: tạm xóa đỉnh, tính lại max flow; nếu giá trị giảm 1, chọn đỉnh đó và giữ nó bị xóa. Việc chỉ mã hóa dung lượng bằng $6000 + i$ không bảo đảm thứ tự từ điển, chỉ bảo đảm tổng chỉ số nhỏ, nên cách duyệt kiểm tra lại an toàn hơn.

Section 5.5

Picture (picture). Tính chu vi hợp của các hình chữ nhật. Cách quét: xử lý cạnh ngang và cạnh dọc riêng. Với cạnh ngang, sắp theo tung độ; cạnh bắt đầu làm tăng mức phủ trên đoạn x , cạnh kết thúc làm giảm. Khi mức phủ tại một đơn vị đoạn đổi từ 0 sang 1 hoặc 1 sang 0, chu vi tăng độ dài tương ứng. Làm tương tự cho cạnh dọc. Có thể rời rạc hóa tọa độ và cập nhật mảng mức phủ, hoặc dùng cây đoạn để tăng tốc cập nhật đoạn và lấy độ dài biên thay đổi.

Hidden Passwords (hidden). Tìm vị trí bắt đầu của phép quay từ điển nhỏ nhất. Nhân đôi chuỗi rồi dùng thuật toán hai con trỏ Booth: giữ hai ứng viên i, j và độ dài khớp k . Nếu $s[i + k] = s[j + k]$, tăng k ; nếu $s[i + k] > s[j + k]$, mọi vị trí từ i đến $i + k$ không tối ưu nên đặt $i = i + k + 1$; ngược lại dời $j = j + k + 1$. Nếu hai con trỏ trùng thì tăng con trỏ sau. Khi một con trỏ vượt n , đáp án là $\min(i, j)$. Nguồn ghi chú cũng nêu cách so sánh trực tiếp có thể qua test yếu nhưng worst-case $O(n^2)$.

Two Five (twofive). Các từ hợp lệ là cách điền $A..Y$ vào bảng 5×5 sao cho hàng và cột tăng. Cần chuyển giữa thứ tự từ điển và mã số. Trọng tâm là hàm đếm số cách hoàn thành khi một số chữ cái đã bị cố định. Dùng memo theo trạng thái (a, b, c, d, e) , số ô đã điền ở từng hàng, luôn thỏa $5 \geq a \geq b \geq c \geq d \geq e \geq 0$. Ở bước đặt chữ cái kế tiếp, thử các hàng còn chỗ mà việc thêm ô vẫn giữ dạng Young diagram và không mâu thuẫn vị trí cố định. Để mã hóa từ, cộng số cách của mọi lựa chọn chữ cái nhỏ hơn tại vị trí hiện tại; để giải mã số, thử từng chữ cái và trừ dần số cách cho đến khi rơi vào nhánh chứa mã cần tìm.

Chương 6

Section 6.1

Postal Vans (vans). Bài cần đếm đường đi qua mọi điểm trong lưới đặc biệt, kết quả lớn nên dùng big integer. Nguồn ghi chú đưa ra hai trạng thái chính: $a[i]$ là số cấu hình kết thúc ở cột i với hai điểm góc trái trên/trái dưới liên thông theo dạng cuối thứ nhất, $b[i]$ là số cấu hình dạng cuối thứ hai. Công thức tối giản:

$$b[i] = a[i - 1] + b[i - 1],$$

$$a[i] = a[i - 2] + 2 + 2 \sum_{t=3}^{i-1} b[t],$$

với các giá trị đầu được khởi tạo theo hình dạng cơ sở. Khi cài đặt, duy trì tổng tích lũy của các $b[t]$ để tính mỗi cột trong $O(1)$; toàn bộ là $O(n)$ phép cộng số lớn.

A Rectangular Barn (rectbarn). Tìm hình chữ nhật trống lớn nhất trong lưới có một số ô hỏng. Với mỗi ô, tính $height[j]$, số ô trống liên tiếp phía trên tại cột j . Mỗi hàng trở thành bài histogram lớn nhất: dùng stack đơn điệu để tìm, cho từng cột, đoạn trái/phải mà chiều cao hiện tại là nhỏ nhất, rồi cập nhật diện tích $height[j] \cdot width$. Đây là phiên bản gọn của DP h, l, r trong ghi chú: h là chiều cao, l, r là biên mở rộng trái/phải. Dùng rolling array để phù hợp giới hạn bộ nhớ.

Cow XOR (cowxor). Đặt prefix xor $px[0] = 0, px[i] = a_1 \oplus \dots \oplus a_i$. XOR của đoạn $[l, r]$ là $px[r] \oplus px[l - 1]$. Để tối đa hóa với mỗi $px[i]$, lưu các prefix trước đó trong trie nhị phân từ bit cao xuống thấp; khi truy vấn, luôn đi nhánh bit đối lập nếu có để tạo bit 1. Lưu chỉ số prefix ở lá để khôi phục đoạn. Khi hòa theo giá trị, áp dụng đúng quy tắc đề về điểm đầu và điểm cuối. Vì giá trị chỉ khoảng 21 bit, mỗi thao tác là $O(21)$, tổng $O(n)$.